Final Year Project Report

SmartTrade Agentic AI System

[Your Name]
[Department / University]

[YYYY-MM-DD]

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Purpose of the Project

This report documents the SmartTrade Agentic AI System and serves as the project's requirements and scope reference within the Final Year Project (FYP) context. Its purpose is to define the system objectives, scope, constraints, development timeline, and requirements prior to implementation, clarifying what the system is expected to achieve and the boundaries under which it will operate.

It identifies and documents the business requirements, user requirements, functional requirements, and non-functional requirements in a structured and clear manner, providing a shared reference point for the academic supervisor, evaluators, and system developers.

Because the project is bounded by the FYP timeline, academic constraints, and limited resources, this document also establishes feasibility-focused boundaries and acts as a baseline for the system development life cycle, validation, academic assessment, and future enhancements.

1.2 Intended Audience

This report is intended for multiple stakeholders involved in the development, supervision, evaluation, and validation of the system. The primary audience includes the academic supervisor, FYP evaluators, and examiners, who will review the report for technical feasibility, completeness, and alignment between the implemented system and intended artifacts such

as design, testing, and requirements.

It is also intended for the project developers, who will use the specified requirements and constraints to guide system design and implementation. In addition, the report may be useful to future researchers who wish to extend, modify, or study SmartTrade as a reference example of AI-assisted trading platforms.

Overall, this report serves as a formal communication artifact to ensure shared understanding of the system boundaries, objectives, expectations, and requirements among stakeholders, reducing ambiguity and supporting structured development.

1.3 Problem Statement

Retail traders often have trading intuitions and strategies in mind, but face a "syntax cliff" when translating those ideas into precise, executable rules. Existing platforms typically require users to express logic through code, rigid rule-builders, or fragile middleware integrations. As a result, strategies are frequently incomplete, inconsistent, or incorrectly implemented, which creates a trust deficit and increases the risk of user harm.

In addition, many trading automation workflows do not provide sufficient transparency and traceability (e.g., clear explanation of interpreted rules, validation evidence, and audit logs). This makes it difficult for users and academic evaluators to verify that the implemented strategy behavior matches the intended strategy description.

Therefore, there is a need for a system that can translate natural language trading intention into structured and verifiable trading logic, enforce safety constraints, and provide evaluation artifacts such as backtesting reports and traceable logs, while remaining feasible within the academic FYP timeline and resource constraints.

1.4 Objectives

- Enable users to express trading strategies in natural language and convert them into structured, verifiable trading logic.
- Provide automated validation and safety/risk controls to reduce the likelihood of unsafe or inconsistent strategies.

- Support evaluation through automated backtesting, reporting, and traceability artifacts suitable for academic assessment.

1.5 Scope and Limitations

This report defines the scope of the SmartTrade Agentic AI System by specifying what the system will and will not do within the constraints of the academic FYP timeline and the MVP. It outlines functional capabilities, non-functional expectations, system boundaries, and operational limitations required to deliver a working and assessable system.

The scope is intentionally constrained to focus on translating natural language trading intuition into executable and verifiable trading logic, supported by automated backtesting, parameter optimization, and enforced risk controls. Features that would introduce excessive complexity, implementation risk, or resource burden are explicitly excluded, such as high-frequency trading (HFT), real-time machine learning, and broker custody.

By defining clear boundaries and constraints, the project remains feasible, testable, and aligned with both academic evaluation criteria and real-world relevance.

1.6 Report Organization

This report is organized into structured chapters to present the system requirements in a clear, logical, and traceable manner. Each chapter builds on the previous to progressively define the system purpose, scope, behavior, timeline, and constraints.

Chapter 1 introduces the background, purpose, motivation, and stakeholders. Chapter 2 describes the overall system, including product perspective, user characteristics, and operating environment. Subsequent chapters define business requirements, user requirements, functional requirements, and non-functional requirements for the SmartTrade Agentic AI System.

Later chapters include modeling and analytical artifacts such as use case, activity, sequence, and class diagrams, as well as verification and validation artifacts to ensure logical and technical consistency. These artifacts help clarify user interactions and reduce ambiguity in the requirements.

1.7 Definitions, Acronyms, and Abbreviations

Definitions

Term	Definition
SmartTrade	The SmartTrade Agentic AI System that converts natural-language trading intent into a structured strategy specification, validates it for safety/completeness, and evaluates it via backtesting.
Agentic AI	An AI-driven workflow that can plan a multi-step process (e.g., ask clarification questions, validate constraints, run backtests) under explicit rules and guardrails.
Strategy Specification	A structured representation of a strategy (instrument, timeframe, indicators, entry/exit logic, risk rules, parameters) that can be backtested and reviewed.
Clarification Question	A targeted question asked when the user input is ambiguous, incomplete, or contradictory, to ensure the resulting strategy is executable and testable.
Risk Gate	A set of checks that must pass before a strategy is considered eligible for execution (e.g., max drawdown threshold, position size limits, stop-loss requirement).
Backtesting	Running a strategy over historical market data to produce performance metrics and trade-by-trade evidence.
Audit Log	A traceable record of user inputs, clarifications, validations, decisions, and outputs used to support transparency and academic evaluation.
Kill-Switch	A user-controlled mechanism that stops or prevents trade execution immediately.

Acronyms

Acronym	Full Form
LLM	Large Language Model
HFT	High-Frequency Trading

MT5	MetaTrader 5
NLP	Natural Language Processing
API	Application Programming Interface
RTM	Requirements Traceability Matrix

Abbreviations

Abbreviation	**Description**
SL	Stop Loss
TP	Take Profit
DD	Drawdown

1.8 References

The following references are used as supporting inputs for requirements, scope, and validation planning:

- Project proposal and FYP scope synthesis documents (internal project artifacts).
- Market research and competitive analysis documents included in the project repository.
- Platform experience analysis of retail trading terminals (internal project artifact).
- MetaTrader 5 (MT5) platform documentation and relevant broker/API documentation (for integration assumptions).
- Backtesting and trading-system evaluation references used to justify metrics (e.g., drawdown, risk limits, and reporting).

Chapter 2

Overall System Description

2.1 Product Perspective

The SmartTrade Agentic AI System is a bounded, web-accessible orchestration layer that sits between a human trader and existing trading execution and analysis platforms. It is conceived as a translation, validation, and safety engine rather than as a broker, exchange, or market data provider. The system's role is to accept a trader's strategy expressed in natural language, convert that intent into inspectable, platform-compatible trading logic, validate the logic through controlled simulation, and transmit verified trading instructions to an external execution terminal or produce artifacts for user review. SmartTrade does not hold custody of funds, perform order matching, or provide clearing services; all live execution remains under the control of the connected trading client or broker.

SmartTrade is designed to interoperate with established retail trading ecosystems. For analysis and visual strategy representation the system targets TradingView (Pine Script) and, for execution-oriented strategies, MetaTrader 5 (MQL5). This dual-target approach separates analysis/visualization from execution responsibilities and leverages existing platform capabilities to reduce scope and regulatory complexity. Users therefore receive both an analysis representation (for inspection and charting) and, where appropriate, an execution artifact compatible with the user's trading terminal.

Conceptually, SmartTrade moves beyond static “no-code” builders and passive chatbots toward an agentic model: the system reasons about user intent, consults grounded language-to-code knowledge, requests clarifications when inputs are ambiguous, and validates outputs before they are presented. To mitigate known risks with unconstrained language models,

the generation process is supported by grounded references and curated code templates so that produced scripts adhere to the target platforms' syntax and common safety patterns. By making generated logic visible and providing plain-English explanations for the mapping from intent to code, the system embraces a "grey-box" transparency model that prioritizes user understanding and trust.

Within the overall trading automation landscape, SmartTrade is purposely scoped as an academic prototype: feature selection and performance claims are constrained to what can be reliably designed, implemented, and evaluated within an FYP timeframe. The system therefore focuses on retail timeframes (non-HFT), a limited set of asset classes (e.g., Forex and indices via supported terminals), deterministic snapshot or short-lookback strategy logic, rigorous backtesting with stress scenarios, and strict risk guardrails (mandatory stop losses, position sizing limits, and an emergency stop mechanism). These scope boundaries are intended to ensure demonstrable, safe, and explainable end-to-end behavior that can be validated in an academic setting.

2.2 Product Functions

SmartTrade provides the following product functions:

- **Natural-language trading strategy input:** The system accepts trading strategy descriptions, risk constraints, and preferences expressed in natural language by the user.
- **Ambiguity detection and clarification:** The system identifies incomplete, vague, or ambiguous strategy descriptions and engages the user in a clarification process before further processing.
- **Multi-platform strategy translation:** The system translates validated strategy descriptions into readable, platform-compatible trading logic so for:
  - **MetaTrader 5 (MQL5)** for execution-oriented strategies, and
  - **TradingView (Pine Script)** for analysis, visualization, and strategy representation.
- **Platform-aware code validation:** The system performs automated checks to ensure that generated MQL5 and Pine Script code conforms to platform syntax rules and avoids common logical and safety-related errors.

- **Historical backtesting and performance evaluation:** The system evaluates generated strategies using historical market data to produce performance metrics, trade logs, and risk indicators prior to any execution or demonstration.
- **Automated risk guardrail enforcement:** The system enforces mandatory risk constraints—such as position sizing limits, stop-loss requirements, and maximum drawdown thresholds—across all supported platforms.
- **Explainable strategy representation (Grey Box):** The system provides a human-readable explanation that maps the user's original intent to the generated MQL5 or Pine Script logic, improving transparency and user trust.
- **User review, approval, and iteration:** The system allows users to review generated code and backtesting results, approve strategies, or request refinements that restart the validation flow.
- **Strategy status reporting and audit visibility:** The system presents strategy evaluation status, validation outcomes, and historical records to support traceability and academic verification.
- **Strategy artifact export:** The system enables users to export validated strategy artifacts, including Pine Script or MQL5 files and associated performance reports, for external review or controlled deployment.

2.3 User Classes and Characteristics

This section identifies the primary users who directly interact with the SmartTrade Agentic AI System. Only user classes that actively participate in the system's core workflow are included. No auxiliary, administrative, or external observer roles are defined.

2.3.1 Retail Trader

The Retail Trader is a primary system user who utilizes the SmartTrade system to design, evaluate, and review algorithmic trading strategies without directly writing platform-specific code.

This user interacts with the system by:

- Providing trading strategies and constraints using natural language
- Reviewing generated trading logic for MetaTrader 5 (MQL5) and TradingView (Pine Script)
- Analyzing backtesting results and performance metrics
- Approving strategies for export or requesting iterative refinements

Characteristics of this user class include:

- Practical knowledge of financial markets and trading concepts
- Limited or no experience with programming or algorithmic trading languages
- Dependence on system-provided explanations, validation, and risk safeguards
- Primary interaction through a web-based user interface

The system is designed to reduce technical complexity for this user while maintaining transparency and control over strategy outcomes.

2.3.2 System Administrator

The System Administrator represents the operational stakeholder responsible for maintaining system integrity during development, testing, and controlled demonstrations. This user does not design strategies for trading purposes, but manages monitoring, safety thresholds, and auditability to ensure the system remains reliable and defensible in an academic context.

This user interacts with the system by:

- Monitoring system-level metrics related to orchestration performance and signal/execution latency.
- Verifying the operational status of grounded knowledge sources and toolchains used to support accurate and safe strategy generation.
- Confirming data consistency between historical validation datasets and reference/live market data sources used for evaluation.

- Managing logical isolation of user data (e.g., credentials, strategies, and activity logs) across multiple system users.
- Configuring global risk thresholds and enforcing system-wide halts when critical safety conditions are detected.
- Retrieving immutable audit records linking strategy outputs and (where applicable) execution events back to original user prompts and strategy versions.
- Monitoring inputs for abuse patterns that attempt to bypass validation and enforced safety constraints.

Characteristics of this user class include:

- High emphasis on system integrity, safety enforcement, and traceability rather than profitability.
- Requirement for reliable logs, reproducible runs, and auditable evidence for FYP evaluation.
- Focus on defensive operation (safe-state defaults, kill-switch readiness, and incident visibility).

2.3.3 User Class Scope Boundary

No additional user classes are supported by the system. Roles such as brokers, regulators, or platform operators are outside the scope of this project and do not directly interact with the system's primary functionality.

2.4 Operating Environment

The SmartTrade Agentic AI System is designed to operate within a cross-platform, containerized operating environment to ensure consistency, portability, and reliability across different host systems.

All core system components—including strategy interpretation, code generation, validation, and backtesting—are executed inside Docker containers. This approach allows the system

to run consistently on multiple host operating systems such as Windows, Linux, and macOS without requiring environment-specific modifications.

The containerized environment encapsulates:

- The backend orchestration logic
- Language model interaction and prompt execution
- Strategy validation and backtesting services
- Platform-specific tooling required for MetaTrader 5 (MQL5) and TradingView (Pine Script) compatibility

By isolating system dependencies within containers, the operating environment minimizes configuration conflicts, dependency mismatches, and host-specific failures. This design choice supports reproducibility, simplifies deployment, and ensures that system behavior remains stable throughout development, testing, and demonstration phases.

External trading platforms such as MetaTrader 5 and TradingView are treated as integrated external systems rather than native runtime environments. Interaction with these platforms is performed through generated strategy artifacts and controlled execution interfaces, ensuring that platform-specific constraints do not compromise the stability of the core system.

The dockerized operating environment is selected to align with the project's academic constraints, limited development timeline, and requirement for demonstrable end-to-end functionality.

2.5 Design and Implementation Constraints

The SmartTrade Agentic AI System is developed under a set of explicit design and implementation constraints. These constraints are intentionally defined to ensure technical feasibility, academic validity, and controlled system complexity within the Final Year Project (FYP) timeline.

2.5.1 Scope and Delivery Constraints

The SmartTrade Agentic AI System is intentionally designed with a constrained functional scope to ensure reliability, clarity of behavior, and verifiable outcomes.

The system prioritizes:

- Deterministic strategy generation over experimental features
- End-to-end workflow completeness over feature breadth
- Validation and safety mechanisms over execution speed or scale

As a result, the system limits supported workflows to a clearly defined set of strategy generation, validation, and controlled execution tasks. Features that would significantly increase architectural complexity without contributing directly to system validation or safety are intentionally excluded.

This constraint ensures that all implemented functionality can be thoroughly tested, verified, and demonstrated under controlled conditions.

2.5.2 Platform and Technology Constraints

The system is constrained to operate with the following trading platforms and scripting environments:

- MetaTrader 5 (MQL5) for automated trading strategy execution
- TradingView (Pine Script) for strategy analysis and visualization

No additional trading platforms, exchanges, or scripting languages are supported. This constraint ensures focused implementation and reduces integration complexity.

2.5.3 Containerization Constraint

All backend components of the system must operate within a Dockerized environment. This constraint ensures:

- Consistent runtime behavior across development and demonstration environments
- Isolation of system dependencies from the host operating system
- Simplified deployment and reproducibility for academic evaluation

Direct execution of core system components on the host operating system without containerization is not permitted.

2.5.4 Performance and Execution Constraints

The system is not designed for high-frequency or ultra-low-latency trading. Therefore:

- Strategies operating on sub-minute or tick-level timeframes are not supported
- The system targets retail-level trading strategies with moderate execution latency
- Real-time optimization or continuous online learning is excluded

These constraints align the system with realistic retail trading use cases and the limitations of a Python-based orchestration layer.

2.5.5 Safety and Risk Constraints

Risk management constraints are mandatory and cannot be bypassed by the user. The system enforces:

- Mandatory stop-loss logic for all generated strategies
- Predefined limits on position sizing and exposure
- A centralized mechanism to halt strategy execution when safety thresholds are breached

These constraints exist to prevent unsafe strategy deployment and to support academic evaluation of responsible system design.

2.5.6 Scope and Feature Constraints

To maintain a manageable scope:

- Live trading with real capital is excluded
- Multi-exchange and multi-asset execution is not supported
- Administrative, broker-level, or regulatory system features are out of scope
- Advanced portfolio-level optimization is not implemented

The system focuses solely on strategy generation, validation, and controlled execution.

2.6 Assumptions and Dependencies

This section outlines the assumptions under which the SmartTrade Agentic AI System is designed to operate and the external dependencies required for correct system behavior. These assumptions and dependencies define the operational context of the system and establish conditions that must hold true for successful execution.

2.6.1 Assumptions

The system is designed based on the following assumptions:

- Users possess basic knowledge of trading concepts such as indicators, risk management, and order types.
- Users provide strategy descriptions in clear and meaningful natural language.
- The generated strategies are used for analysis, validation, or controlled execution purposes, not for unrestricted live trading.
- Market data used for backtesting and evaluation is assumed to be accurate, complete, and free from corruption.
- The host environment supports containerized execution using Docker.

- Generated strategy code is reviewed by the user before acceptance or deployment.

These assumptions define the expected operating conditions and user behavior required for correct system operation.

2.6.2 Dependencies

The SmartTrade Agentic AI System depends on the following external systems, tools, and services:

- **MetaTrader 5 platform:** Required for executing and validating MQL5-based trading strategies.
- **TradingView platform:** Required for generating and evaluating Pine Script strategies for chart-based analysis.
- **Large Language Model (LLM) service:** Required for interpreting natural language strategy descriptions and generating platform-specific code.
- **Containerization platform (Docker):** Required to ensure consistent runtime environments across development and deployment systems.
- **Historical market data sources:** Required for strategy backtesting and performance evaluation.
- **Internet connectivity:** Required for accessing LLM services, retrieving documentation references, and synchronizing external data sources.

Failure or unavailability of these dependencies may limit or prevent system functionality.

Chapter 3

System Features and Requirements

3.1 Overview of System Features and Requirements

This chapter defines the functional and non-functional requirements of the SmartTrade Agentic AI System. The requirements specified in this chapter describe what the system must do to meet its intended objectives and user needs, without prescribing how those requirements will be implemented.

The system features and requirements are derived from the defined system scope and operating constraints, the identified primary user classes (Retail Trader and System Administrator), and the need to ensure safety, transparency, and validation in AI-assisted trading systems.

Requirements are organized hierarchically into the following categories:

- **Business Requirements (BRs):** high-level goals and value the system provides.
- **User Requirements (URs):** what users expect to achieve through interaction with the system.
- **Functional Requirements (FRs):** the system's observable behavior and core operations, ranging from natural-language interpretation and code generation to validation, reporting, and artifact export.
- **Non-Functional Requirements (NFRs):** performance, reliability, security, usability, and other quality constraints the system must satisfy.

Each requirement is written in clear, testable language and assigned a unique identifier to support traceability, validation, and future system evolution. Requirements are designed to be atomic where possible and aligned with FYP evaluation criteria.

3.2 Business Requirements

This section defines the high-level business objectives that the SmartTrade Agentic AI System must satisfy to be considered successful. These requirements describe the value the system provides to its intended users and stakeholders (with emphasis on the Retail Trader / "Frustrated Transitioner" persona) and establish the strategic justification for system development.

3.2.1 BR-1: End-to-End Strategy Automation

The system shall provide a unified pipeline that transforms unstructured natural-language trading descriptions into executable, platform-specific algorithmic logic.

Rationale: This addresses the primary gap where retail traders possess strategic intent but lack the capability to implement and automate that intent in platform-specific code.

3.2.2 BR-2: Reduction of Technical Dependency (Syntax-Cliff Mitigation)

The system shall function as an abstraction layer that reduces the need for users to write, debug, or manually manage proprietary trading code syntax (e.g., MQL5 or Pine Script).

Rationale: By internalizing technical complexity, the system allows users to focus on strategy logic and risk intent rather than programming mechanics.

3.2.3 BR-3: Mandatory Strategy Validation Before Acceptance or Deployment

The system shall validate generated trading strategies through historical backtesting and rule-based checks before allowing acceptance, export, or controlled deployment.

Rationale: Mandatory validation reduces "trust debt" by preventing deployment of incomplete, inconsistent, or statistically unsound logic and by increasing user confidence in the generated artifacts.

3.2.4 BR-4: Enforcement of Risk and Safety Controls

The system shall inject and enforce mandatory risk management constraints—including position sizing caps, stop-loss requirements, and drawdown limits—on every generated strategy.

Rationale: Uncontrolled automated strategies can result in excessive losses. Enforced guardrails protect users from unsafe configurations and unmanaged downside exposure.

3.2.5 BR-5: Explainability and Transparency (Grey-Box Mandate)

The system shall provide human-readable, plain-English explanations of generated code and trading logic to ensure users can understand, review, and verify the mapping from intent to executable strategy artifacts.

Rationale: Transparency addresses the "black-box" trust deficit and supports user learning and intellectual ownership of the strategy.

3.2.6 BR-6: Multi-Platform Ecosystem Compatibility

The system shall support the generation and validation of trading logic compatible with industry-standard platforms, specifically MetaTrader 5 (MQL5) and TradingView (Pine Script).

Rationale: Supporting widely adopted platforms ensures the system outputs are practical, comparable, and relevant for retail and semi-professional trading workflows.

3.2.7 BR-7: Integrated Lifecycle Demonstrability

The system shall support a complete end-to-end workflow that can be demonstrated and evaluated as a single integrated process, including natural-language input, strategy genera-

tion, validation through backtesting, risk enforcement, and user review.

Rationale: For FYP assessment, the system must demonstrate coherent end-to-end behavior rather than isolated components.

3.2.8 BR-8: Operational Fail-Safe and Safe-State Behavior

The system shall include an emergency kill-switch and fail-safe protocols that default the system to a safe state (halting execution and blocking further deployment) in the event of a critical failure or a risk threshold breach.

Rationale: Safe-state behavior is a baseline requirement for responsible trading automation and supports academic evaluation of safety-oriented design aligned with relevant market integrity and risk-management guidance.

3.3 User Requirements

This section specifies the tasks each defined user class must be able to perform when interacting with the SmartTrade Agentic AI System. These requirements are derived from validated market pain points, including strategy translation errors, integration fragility, and lack of system transparency.

3.3.1 Retail Trader Requirements

UR-1.1: Conversational Strategy Input
The user shall be able to describe entry conditions, exit rules, and risk preferences using unstructured natural language through the system interface.

UR-1.2: Interactive Ambiguity Resolution
The user shall participate in system-initiated clarification interactions to resolve ambiguous or subjective trading terms prior to strategy generation.

UR-1.3: Strategy Ownership Review
The user shall receive a plain-English explanation of the generated trading logic to verify alignment with their intended strategy.

UR-1.4: Strategy Robustness Review
The user shall be able to review performance metrics, including profit, drawdown, and risk indicators, derived from historical and stress-test evaluations.

UR-1.5: Strategy Execution Control
The user shall be able to manually activate, pause, or terminate any strategy instance through the system dashboard.

UR-1.6: Automated Risk Feedback
The user shall be notified whenever the system applies mandatory risk controls, including position sizing limits or automatically injected stop-loss rules.

UR-1.7: Multi-Platform Strategy Export
The user shall be able to export validated trading logic in a format compatible with MetaTrader 5 (MQL5) or TradingView (Pine Script).

3.3.2 System Administrator Requirements

UR-2.1: Execution Latency Monitoring
The user shall be able to monitor system-level metrics related to signal transmission and execution latency.

UR-2.2: Strategy Generation Integrity Monitoring
The user shall be able to verify the operational status of knowledge sources used to ensure accurate and grounded strategy generation.

UR-2.3: Data Consistency Verification
The user shall be able to confirm that historical data used for validation aligns with live or reference market data sources.

UR-2.4: User Data Isolation Management
The user shall ensure that trader-specific data, including credentials and activity logs, remains logically isolated across all system users.

UR-2.5: System-Wide Risk Control Management
The user shall be able to configure global risk thresholds that trigger an immediate halt of all active strategy execution.

UR-2.6: Audit Trail Access
The user shall be able to retrieve immutable audit records linking executed trades to origi-

nating user prompts and generated strategy versions.

UR-2.7: Security and Input Abuse Monitoring
The user shall be able to monitor system inputs for malicious or abnormal usage patterns that may attempt to bypass enforced safety constraints.

3.4 Functional Requirements

This section defines the functional requirements of the SmartTrade Agentic AI System. These requirements specify the system behaviors necessary to support an end-to-end workflow for natural-language strategy creation, validation, risk enforcement, and controlled execution. The requirements are organized according to the system's operational lifecycle.

3.4.1 Strategy Input and Interpretation

FR-01: Natural-Language Strategy Intake

Description: The system shall accept unstructured natural-language input describing trading intent, constraints, and preferences through a conversational interface.

Priority: Critical

Rationale: Enables non-programmatic users to express trading strategies without requiring formal syntax knowledge.

Acceptance Criteria: The system successfully accepts free-form text input and initiates intent interpretation without requiring predefined templates.

FR-02: Interactive Ambiguity Resolution (Clarification Loop)

Description: The system shall identify ambiguous, incomplete, or vague strategy descriptions and initiate a clarification dialogue with the user before proceeding.

Priority: Critical

Rationale: Prevents incorrect or hallucinated logic generation and enforces deterministic interpretation.

Acceptance Criteria: If essential parameters are missing or unclear, the system pauses execution and requests clarification from the user.

FR-03: Semantic Intent Structuring

Description: The system shall convert validated user input into a structured internal representation separating strategy logic, execution rules, and risk parameters.

Priority: High

Rationale: Provides a formal intermediate representation required for safe code generation and validation.

Acceptance Criteria: User input is transformed into a structured model suitable for downstream generation and analysis.

3.4.2 Agentic Strategy Generation

FR-04: RAG-Enhanced Code Generation

Description: The system shall generate trading strategy source code using a Retrieval-Augmented Generation (RAG) approach grounded in validated platform-specific templates.

Priority: Critical

Rationale: Reduces hallucination risk and improves syntactic correctness of generated code.

Acceptance Criteria: Generated code conforms to MetaTrader 5 (MQL5) or TradingView (Pine Script) syntax standards.

FR-05: Automated Code Validation and Regeneration

Description: The system shall automatically analyze generated code for syntactic or structural errors and regenerate corrected code if issues are detected.

Priority: High

Rationale: Eliminates the need for users to debug generated output.

Acceptance Criteria: Detected syntax errors trigger automatic correction without user intervention.

FR-06: Cross-Platform Strategy Generation

Description: The system shall generate functionally equivalent trading strategies compatible with both MetaTrader 5 and TradingView from a single user intent.

Priority: High

Rationale: Supports multi-platform deployment and analysis workflows.

Acceptance Criteria: The system produces compilation-ready MQL5 and Pine Script outputs for the same strategy logic.

3.4.3 Validation, Risk, and Safety

FR-07: Historical Strategy Backtesting

Description: The system shall evaluate generated strategies using historical market data and produce performance metrics.

Priority: Critical

Rationale: Ensures strategies are logically coherent and measurable prior to execution.

Acceptance Criteria: The system generates a backtest report including net profit, drawdown, and trade history.

FR-08: Strategy Stress and Robustness Testing

Description: The system shall assess strategy robustness under adverse or stressed market conditions.

Priority: High

Rationale: Identifies fragile strategies and reduces the backtest-reality gap.

Acceptance Criteria: The system provides robustness or stress-test results alongside standard backtest metrics.

FR-09: Mandatory Risk Control Enforcement

Description: The system shall enforce predefined risk management constraints, including position sizing limits and mandatory stop-loss rules.

Priority: Critical

Rationale: Prevents unsafe strategy deployment due to missing or excessive risk exposure.

Acceptance Criteria: If risk controls are absent, the system injects default protective rules and informs the user.

FR-10: Explainable Validation Feedback

Description: The system shall provide human-readable explanations of validation outcomes and applied safety constraints.

Priority: High

Rationale: Improves transparency, trust, and academic evaluability.

Acceptance Criteria: Validation reports clearly explain why a strategy passed or failed and what constraints were applied.

3.4.4 Execution and Orchestration

FR-11: Strategy Lifecycle Management

Description: The system shall manage strategies through defined lifecycle states, including draft, validated, approved, paused, and terminated.

Priority: High

Rationale: Prevents accidental execution of unvalidated strategies.

Acceptance Criteria: A strategy cannot enter an executable state without satisfying validation requirements.

FR-12: Execution Platform Integration

Description: The system shall support controlled integration with external trading platforms for signal transmission and execution.

Priority: Critical

Rationale: Enables real-world applicability while maintaining execution control.

Acceptance Criteria: Validated strategies can transmit execution signals to the configured platform environment.

FR-13: Emergency Strategy Termination (Kill Switch)

Description: The system shall provide a mechanism to immediately halt all active strategies upon user or system trigger.

Priority: Critical

Rationale: Required for operational safety and failure containment.

Acceptance Criteria: Activating the kill switch halts all active strategies and issues appropriate stop commands.

3.4.5 Traceability and System Integrity

FR-14: Strategy Artifact Export and Audit Logging

Description: The system shall allow export of generated strategy artifacts and maintain an immutable audit trail linking prompts, strategy versions, and execution events.

Priority: High

Rationale: Supports academic verification and post-mortem analysis.

Acceptance Criteria: Auditors can trace any execution event back to the originating user input and strategy version.

FR-15: Multi-User Data Isolation

Description: The system shall enforce strict separation of user data, including strategies, credentials, and execution logs.

Priority: High

Rationale: Prevents data leakage and ensures system integrity in multi-user environments.

Acceptance Criteria: Unauthorized access to another user's data is blocked by the system.

3.5 Non-Functional Requirements

This section specifies the quality attributes and operational constraints under which the SmartTrade Agentic AI System must operate. These requirements define *how well* the system performs its functions rather than *what* functions it performs. The non-functional requirements are essential to ensure system reliability, operational safety, usability, and academic feasibility within the defined project scope.

NFR-01: Performance — Strategy Interpretation Response Time
Requirement: The system shall process and interpret a natural-language trading strategy input and return a structured intent representation within **5 seconds** under normal operating conditions.
Rationale: The system operates as a conversational agent. Excessive latency at the input stage would degrade usability and break the interactive clarification workflow defined in the functional requirements.

NFR-02: Performance — Execution Signal Latency
Requirement: The system shall transmit trade execution signals from the Python backend orchestration layer to the MetaTrader 5 terminal within **2 seconds** (2000ms) in a controlled execution environment.
Rationale: Timely execution is critical for retail automated trading to minimize slippage. This metric supports the credibility of the bi-directional execution bridge.

NFR-03: System Reliability
Requirement: The system shall successfully complete the full strategy lifecycle—from initial generation through validation, backtesting, and final reporting—without data loss or process hangs in at least **95%** of execution runs.

Rationale: The system consists of multiple dependent stages. High end-to-end reliability is required to prevent partial failures that could invalidate results or mislead users.

NFR-04: Safety and Fail-Safe Behavior
Requirement: The system shall immediately transition all active trading strategies to a **safe halted state** (stopping new orders and attempting to close open ones) whenever critical failures are detected, including execution-bridge disconnection, validation service failure, or risk-threshold violations.
Rationale: Automated trading systems must prioritize capital protection. A deterministic fail-safe mechanism prevents unintended execution under failure conditions.

NFR-05: Security — Data Isolation
Requirement: The system shall enforce strict logical separation of user data, ensuring that strategy artifacts, execution logs, API keys, and credentials are accessible only to the authenticated session that owns them.
Rationale: Data isolation prevents unauthorized access and cross-user leakage in a multi-user web environment.

NFR-06: Usability (Grey-Box Transparency)
Requirement: The system shall present generated strategies, validation reasoning, and backtesting outcomes in plain, human-readable language suitable for users without programming expertise (approximately 10th-grade reading level).
Rationale: The primary user is a retail trader transitioning from manual to algorithmic trading. Clear explanations are necessary to establish trust and reduce "black box" anxiety.

NFR-07: Scalability (Controlled Scope)
Requirement: The system architecture shall support concurrent strategy generation and validation requests from multiple distinct users within a Dockerized environment without significant degradation of core functionality or latency.
Rationale: While massive retail scaling is out of scope, the prototype must demonstrate basic concurrency capability.

NFR-08: Maintainability (Modular Design)
Requirement: The system shall be engineered with a modular architecture such that changes to the strategy generation logic (e.g., updating RAG prompts) can be made without requiring code modifications to the execution bridge, validation engine, or user interface components.
Rationale: Modularity supports maintainability, reduces regression risk, and reflects sound software engineering practice.

NFR-09: Portability
Requirement: The entire backend shall be containerized (e.g., using Docker) to ensure it is deployable on any standard Docker-compatible operating system without complex platform-specific configuration.
Rationale: Containerization ensures consistent behavior across environments and supports reproducibility for academic evaluation.

3.6 Data Requirements

This section defines the data elements required by the SmartTrade Agentic AI System, including the nature of input data, generated outputs, and data storage considerations. These requirements ensure that the system processes, stores, and manages data consistently, securely, and in alignment with its functional objectives.

3.6.1 Input Data Requirements

The system shall accept and process the following categories of input data:

DR-01: User Strategy Input
Unstructured natural-language text describing trading intent, market conditions, and preferences. The input may be incomplete, ambiguous, or high-level and serves as the primary trigger for the strategy generation workflow.
Rationale: The system is designed to interpret intent rather than require rigid, form-based inputs.

DR-02: Clarification Responses
Follow-up user responses generated during the clarification loop. These responses are used to resolve missing or ambiguous strategy parameters.
Rationale: These inputs ensure deterministic strategy interpretation before code generation.

DR-03: Market Data Inputs
Historical OHLCV (Open, High, Low, Close, Volume) price data and symbol metadata (instrument name, timeframe, tick size).
Rationale: Market data is required for backtesting, validation, and robustness analysis.

DR-04: Risk Configuration Data
System-defined default risk parameters (e.g., position sizing caps, stop-loss rules) and optional user-defined risk preferences when provided.
Rationale: Risk configuration supports mandatory risk enforcement regardless of user input quality.

DR-05: System Configuration Data
Platform selection (MetaTrader 5 or TradingView), execution mode (analysis-only or execution-enabled), and environment metadata (e.g., Docker container identifiers).
Rationale: Configuration data ensures correct routing of generation and validation workflows.

3.6.2 Output Data Requirements

The system shall generate and present the following outputs:

DR-06: Generated Strategy Code
Platform-specific, compilation-ready source code artifacts, including MQL5 for MetaTrader 5 and Pine Script for TradingView. Generated code shall include injected risk management logic when applicable.
Rationale: This is the primary tangible artifact produced by the system.

DR-07: Strategy Explanation Artifacts
Plain-English explanations of strategy logic, including a mapping between user intent and generated technical rules.
Rationale: Explanation artifacts support transparency, trust, and user validation.

DR-08: Backtesting and Validation Reports
Performance metrics (e.g., net profit, drawdown, win rate), trade logs and execution summaries, and robustness or stress-testing indicators when applicable.
Rationale: Reports provide evidence of strategy viability prior to acceptance or execution.

DR-09: Execution and Status Feedback
Strategy lifecycle state (e.g., draft, validated, approved, halted) and execution confirmations or error notifications.
Rationale: This feedback keeps users informed of system actions and outcomes.

3.6.3 Data Storage and Retention

The system shall store and manage data according to the following principles:

DR-10: Strategy and Artifact Storage
The system shall store user prompts, generated code versions, and validation reports, and maintain version linkage between prompts, code, and results.
Rationale: Storage is required for traceability, auditability, and academic verification.

DR-11: Execution Logs
The system shall store execution events, timestamps, and system actions, and link logs to the corresponding strategy and user session.
Rationale: Execution logs support debugging, evaluation, and post-mortem analysis.

DR-12: Credential and Sensitive Data Handling
The system shall store sensitive data (e.g., API keys) in encrypted or protected form and shall never store credentials in plain text.
Rationale: This protects users from credential leakage and misuse.

DR-13: Data Retention Policy
The system shall retain strategy artifacts and logs for the duration of the project lifecycle and shall support manual deletion or reset for demonstration purposes.

Rationale: This balances traceability with academic scope and storage constraints.

3.7 External Interface Requirements

This section defines the interfaces through which the SmartTrade Agentic AI System interacts with external actors and platforms. It specifies the necessary communication patterns, data exchange formats, and performance constraints required for seamless integration, without prescribing specific implementation tooling at this stage.

3.7.1 User Interface Requirements (Web Client)

The system shall provide a secure web-based interface accessible via standard modern browsers, serving as the primary interaction point for strategy generation, validation, and control.

EIR-01: Secure Conversational Interface

- The system shall provide a secure (HTTPS/TLS encrypted) text-based interface for submitting natural-language strategy descriptions and handling interactive clarification dialogues.

Rationale: Supports the NLP-driven input workflow while ensuring user intellectual property is protected in transit.

EIR-02: Grey-Box Logic Explanation View

- The system shall display generated strategy explanations and logic summaries in a plain-English, human-readable format, mapping intent to code blocks.

Rationale: Allows non-technical users to verify intent alignment, addressing the "Black Box Trust Deficit."

EIR-03: Backtesting and Visualization Dashboard

- The system shall present backtesting results, equity curves, and risk metrics in a structured, interactive visual layout.

Rationale: Provides transparency and supports data-driven decision-making by non-technical users.

EIR-04: Execution Control & Kill Switch Accessibility

- The system shall provide clearly visible UI controls for managing active strategies, including a prominent "Emergency Kill Switch" to halt execution immediately.

Rationale: Ensures rapid user response during abnormal market conditions, meeting safety mandates.

3.7.2 MetaTrader 5 Interface (Execution Bridge)

The system shall interface with the MetaTrader 5 execution environment via a custom bridge to support automated signal transmission.

EIR-06: Strategy Code Compatibility (MQL5)

- The system shall generate source code files compliant with the MetaTrader 5 (MQL5) language standard, allowing for native compilation within the terminal.

Rationale: Ensures the generated logic is natively runnable within the target execution environment.

EIR-07: Low-Latency Bi-Directional Signal Exchange

- The system shall establish a persistent, bi-directional connection to the MT5 terminal capable of transmitting structured execution signals (e.g., JSON payloads) with minimal latency (meeting NFR-P1 constraints).

Rationale: A persistent connection mechanism is required for automated trading to avoid the overhead of establishing a new connection for every trade.

EIR-08: Market and Account Feedback Loop

- The system shall receive asynchronous updates via the bridge connection containing execution confirmations, order status changes, and real-time account equity data.

Rationale: Required for closed-loop state management, ensuring the backend knows the exact status of every trade.

3.7.3 TradingView Interface (Passive Export)

The system shall interface with TradingView primarily for analysis and visualization purposes via file export.

EIR-09: Pine Script Strategy Export

- The system shall generate source code files compliant with Pine Script v5 syntax that can be directly imported into the TradingView editor.

Rationale: Allows users to utilize TradingView's charting tools for visual backtesting and analysis without requiring live execution.

3.7.4 External AI & Data Service Interfaces

The system shall interface with external services to power its intelligence and Retrieval-Augmented Generation (RAG) capabilities.

EIR-11: LLM Provider Interface

- The system shall communicate with an external Large Language Model provider via a secure API for intent classification and code generation.

Rationale: Enables the core semantic understanding and generative capabilities of the system.

EIR-12: Vector Embedding Store Interface (RAG)

- The system shall interface with a specialized data store capable of indexing and retrieving high-dimensional vector embeddings to provide semantic context for grounded code generation.

Rationale: Critical for the RAG pipeline to ensure generated code uses validated platform syntax, reducing hallucinations.

EIR-13: AI Service Failure Handling

- The system shall implement mechanisms to detect timeouts or errors from external AI APIs and handle unavailability gracefully without compromising system safety.

Rationale: Prevents undefined system behavior when dependent external services suffer temporary outages.

3.8 Out-of-Scope Features

The following features are explicitly excluded from the scope of the SmartTrade Agentic AI System to maintain academic feasibility, operational safety, and architectural clarity within the defined eight-month project timeline.

OS-01: Real-Money Trading on Live Accounts

- The system shall not directly manage or execute trades on live, real-money brokerage accounts. All execution will be controlled in demo or paper trading environments.

Why excluded: Introduces unacceptable financial liability and regulatory risk beyond the scope of an academic prototype.

OS-02: Brokerage Account Management

- The system shall not provide features for user registration with brokers, fund deposits, withdrawals, or KYC verification.

Why excluded: These are regulated functions strictly handled by external financial institutions.

OS-03: Guaranteed Profit or Strategy Optimization

- The system shall not make any claims of guaranteed profitability, nor will it automatically optimize strategies to maximize profit without user input.

Why excluded: Financial markets are probabilistic; performance guarantees are mathematically impossible and ethically misleading.

OS-04: Fully Autonomous, Unsupervised Trading (Black-Box Mode)

- The system shall not deploy or modify active strategies without explicit prior review and manual approval by the user.

Why excluded: Aligns with the human-in-the-loop safety mandate to maintain user oversight and trust.

OS-05: Regulatory Compliance and Financial Advice

- The system output shall not constitute personalized financial advice, nor does the system claim compliance with specific financial regulations (e.g., MiFID II).

Why excluded: Regulatory certification requires specialized legal auditing that is out of scope for an engineering project.

OS-06: Advanced Portfolio Management

- The system shall focus on automating single strategies in isolation and shall not manage multi-asset portfolios, correlation hedging, or cross-account capital allocation.

Why excluded: Adds exponential complexity unrelated to the core problem of bridging the syntax cliff for individual strategies.

OS-07: High-Frequency Trading (HFT)

- The system shall not support ultra-low latency strategies requiring execution speeds under 500ms, nor tick-by-tick arbitrage logic.

Why excluded: Requires specialized hardware infrastructure (co-location, FPGAs) incompatible with standard retail cloud environments.

OS-08: Third-Party Strategy Marketplace

- The system shall not include features for users to buy, sell, share, or rate strategies publicly.

Why excluded: Introduces complex requirements for intellectual property management, payments, and content moderation.

OS-09: Native Mobile Application

- The system shall not provide a native iOS or Android application.

Why excluded: A responsive web-based interface sufficiently supports the defined user workflows for the MVP.

OS-10: Long-Horizon Stateful Logic (Deep Memory)

- The system shall not support strategies requiring complex, multi-day state tracking or memory of events that occurred weeks ago (e.g., comparing to last month's high combined with external events).

Why excluded: Long-horizon state maintenance increases the risk of inconsistent or hallucinated logic; therefore, the system focuses on snapshot market conditions (current and recent candles).

OS-11: Decentralized Finance (DeFi) and On-Chain Execution

- The system shall not support execution on blockchain-based decentralized exchanges (DEXs) or wallet integration (e.g., MetaMask).

Why excluded: The project scope is strictly limited to traditional retail trading platforms (MT5/TradingView) to maintain architectural focus.

3.9 Requirement Traceability Summary